



Fast, Flexible, Open: The New PyTorch Architecture of TensorRT-LLM

Jun Yang | On behalf of the great work from the entire TensorRT-LLM team



Agenda

- Introduction

- API & Ease-of-use

- PyTorch based Modeling&Python runtime

- Performance Optimization

- Future work

Introduction

History of TensorRT-LLM

- Short introduction of TensorRT-LLM© : *TensorRT-LLM provides users with an easy-to-use Python API to define Large Language Models (LLMs) and support state-of-the-art optimizations to perform inference efficiently on NVIDIA GPUs. TensorRT-LLM also contains components to create Python and C++ runtimes that orchestrate the inference execution in performant way.*
- Firstly GH release in 2023.10, as 0.5.0 version.
- Supports GPT-J/LLaMA/Mixtral MLPerf submission since the introduction of LLM workloads into MLPerf
- Used by various industry grade customers, including Amazon/Google/Apple/DeepInfra/Baseten/Tencent/Alibaba/Baidu/PDD/...
- The latest release version is 0.20.0, in monthly cadence, with thorough QA validation
- Also there is weekly based RC release version, with post-merge CI clean.
- The PyTorch re-architecture started since 2024.08, released in [0.17.0](#) in 2025.1 and will become the default flow in 1.0.0 release.
- Initially a GH secondly project, becoming [GH firstly](#) since 2025.3 to be friendly for the community engagement.

Introduction

- Since TensorRT-LLM v0.17, a new **PyTorch** architecture was introduced
- PyTorch flow already as the default flow in the main branch.
- It improves the experience to maintain deployments and extend the system with new models/features while providing SOTA perf on NVIDIA GPUs
 - PyTorch based model implementation, with customized kernels and runtime optimization to assure SOTA performance
 - Modularized Python runtime
 - LLM API that is compatible with HuggingFace checkpoints

Quick Start

- Try PyTorch workflow with a few LoC based on LLM API

```
# Import LLM API
from tensorrt_llm import LLM

# Create a LLM object
llm = LLM(model="./Llama-3.1-8B-instruct")

# Prepare prompts
prompts = [
    "Hi, pls tell me something about reasoning model",
    "Hi, pls tell me something about TensorRT-LLM"
]

# Generate output
output = llm.generate(prompts)
```

TensorRT workflow(<= 0.21 version)

```
# Import LLM API
from tensorrt_llm._torch import LLM

# Create a LLM object
llm = LLM(model="./Llama-3.1-8B-instruct")

# Prepare prompts
prompts = [
    "Hi, pls tell me something about reasoning model",
    "Hi, pls tell me something about TensorRT-LLM"
]

# Generate output
output = llm.generate(prompts)
```

PyTorch workflow(<= 0.21 version)

* the credit for the LLM API idea goes to the vLLM team

```
# Import LLM API(PyTorch as the default flow)
from tensorrt_llm import LLM
# Create LLM object
llm = LLM(model="./Llama-3.1-8B-instruct")
prompts = [
    "Hi, pls tell me something about TensorRT-LLM"
]

output = llm.generate(prompts)

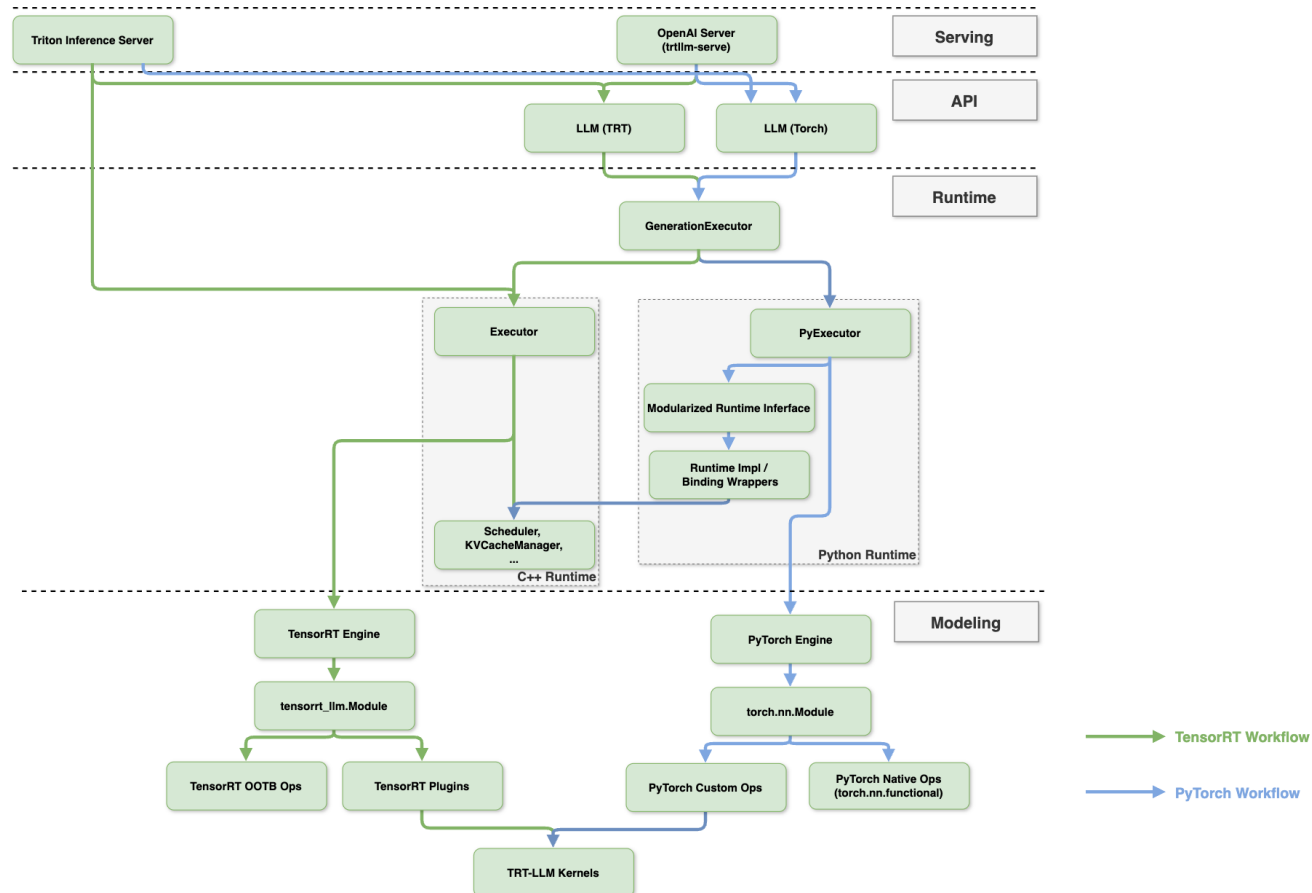
# Import LLM API(TensorRT as the opt-in flow)
from tensorrt_llm._tensorrt_engine import LLM
# Create LLM object
llm = LLM(model="./Llama-3.1-8B-instruct")
prompts = [
    "Hi, pls tell me something about TensorRT-LLM"
]

output = llm.generate(prompts)
```

> 0.21 version

- More examples with additional parameters are available in [examples/llm-api/quick_start_advanced.py](https://github.com/NVIDIA/tensorrt-llm/blob/main/examples/llm-api/quick_start_advanced.py)

Overview of TensorRT-LLM Architecture



* Credit to [github@bobboli](#) for contributing this nice diagram



Agenda

- Introduction

- **API & Ease-of-use**

- PyTorch based Modeling&Python runtime

- Performance Optimization

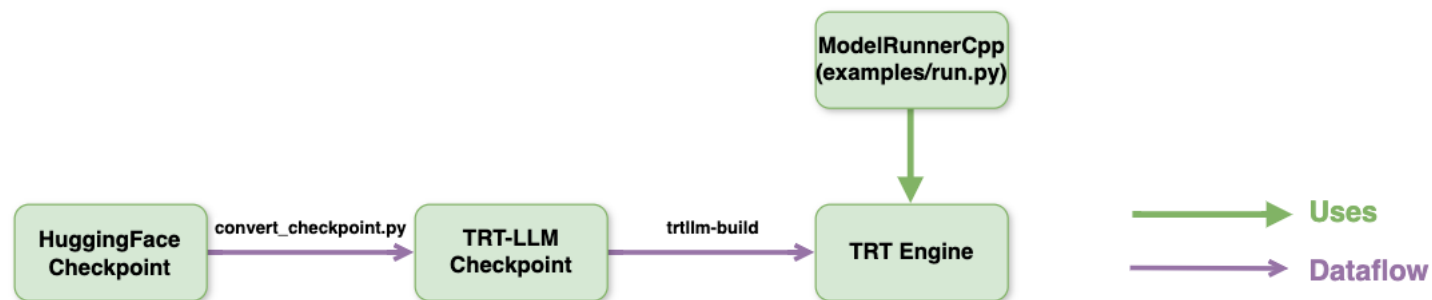
- Future work

User API evolution history

- When we started: Manual TensorRT ckpt conversion/engine build workflow + Python wrapper of TensorRT API for building new models + C++ Runtime
- Later evolution: LLM API based one-stop workflow, under the hood using TensorRT + Python wrapper of TensorRT API for building new models + C++ Runtime with Python binding
- The latest status: LLM API based one-stop workflow, under the hood using PyTorch + PyTorch based model API for building new models + Python Runtime with reusing modularized C++ runtime building blocks

Working with TensorRT requires additional steps

- In TRT workflows, we cannot directly load a HuggingFace format checkpoint and run it
- Two additional steps must be performed by manually running some scripts:
 - **Convert Checkpoint:** Prepare a checkpoint compatible with the model definition in TensorRT-LLM
 - **Build Engine:** Prepare binaries of a runnable TensorRT engine

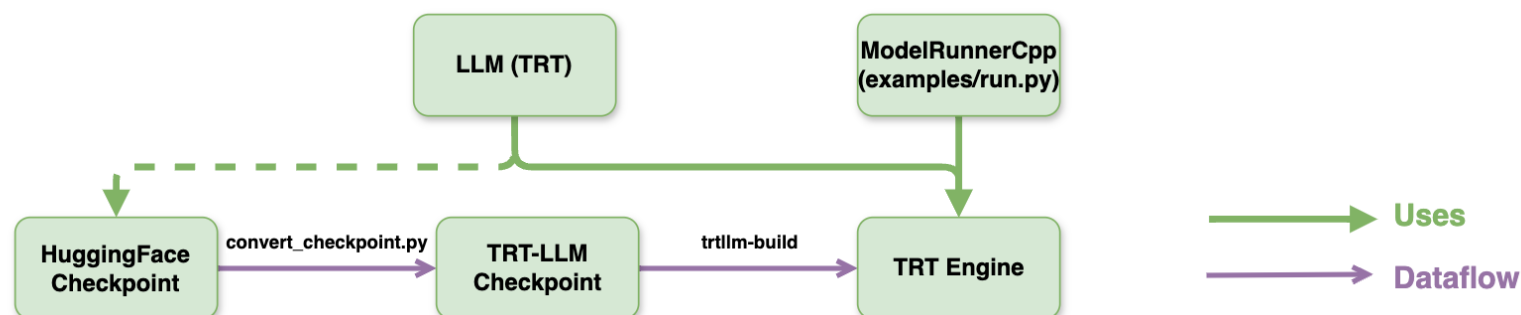


- Not friendly for ML researchers and engineers familiar with the PyTorch/HuggingFace ecosystem

LLM API

w/ TensorRT

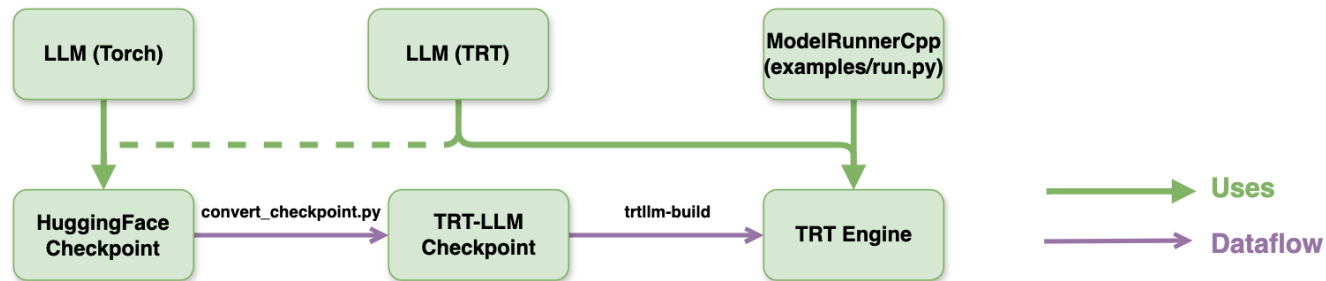
- Manual checkpoint conversion and engine building is cumbersome
 - Especially problematic when the model is large and NFS IO is slow (could be tens of minutes)
 - We need 5 times of IO: Load HF ckpt -> Save TRT-LLM ckpt -> Load TRT-LLM ckpt -> Save TRT engine -> Load TRT engine
 - Also, the parameter lists of `convert_checkpoint.py` and `trtllm-build` are overwhelmingly long and unfriendly for first-time users
- LLM API is introduced to improve ease-of-use
 - We now could provide huggingface ckpts to the API like `LLM(model=ckpt_path)` (indicated using dotted line)
 - However, TRT engine is still under the hood (indicated using solid line)
 - Checkpoint conversion and engine building are still necessary, they happen behind the scene
 - Build params such as `max_batch_size` and `max_num_tokens` are encapsulated into `LLMArgs.BuildConfig`
 - Converted ckpt is in memory and not serialized to disk, but ckpt conversion and engine building are still too heavy for one run



LLM API

w/ PyTorch

- PyTorch workflow introduces PyTorch based modeling support
 - No checkpoint conversion: Similar to vLLM/SGLang or HF transformers, model definition is compatible with the HF ckpts
 - No engine building: Model execution is also based on PyTorch
- The entry point of PyTorch workflow is also the LLM API
 - Much better usability for users
 - Easy to hack





Agenda

- Introduction

- API & Ease-of-use

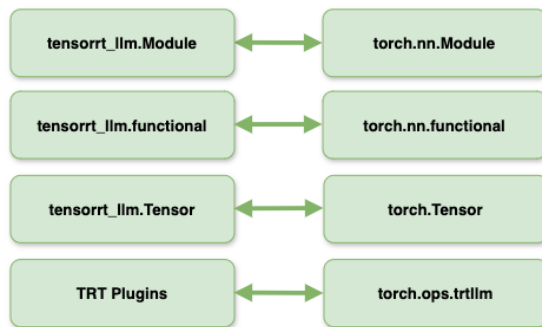
- **PyTorch based Modeling&Python runtime**

- Performance Optimization

- Future work

Develop Models using PyTorch

- Previously, TRT-LLM provides a PyTorch-like way to develop models using TensorRT



- This makes model development much easier compared to using native TensorRT APIs
- However, the dev experience is not as easy as native PyTorch:
 - PyTorch has eager mode execution
 - PyTorch Tensor class is more flexible and powerful
 - Extending PyTorch Ops is easier than TRT Plugins (especially for developers familiar with PyTorch)
- One of the key reasons triggering the PyTorch centric re-architecture

Modules

Building blocks of the model

- TRT-LLM is designed to make the development experience similar to PyTorch at this level
- Two main methods need to be defined for a Module:
 - `__init__`
 - `forward`
- `tensorrt_llm.layers`

```
class RmsNorm(Module):
    def __init__(self,
                 if self.elementwise_affine:
                     self.weight = Parameter(shape=self.normalized_shape, dtype=dtype)
                 else:
                     self.register_parameter('weight', None)

    self.eps = eps
    self.dtype = dtype

    def forward(self, x, normalized_shape=None):
        weight = None if self.weight is None else self.weight.value
        if normalized_shape is None:
            normalized_shape = self.normalized_shape
        return rms_norm(x, normalized_shape, self.num_groups, weight, self.eps)
```

- `tensorrt_llm._torch.modules`

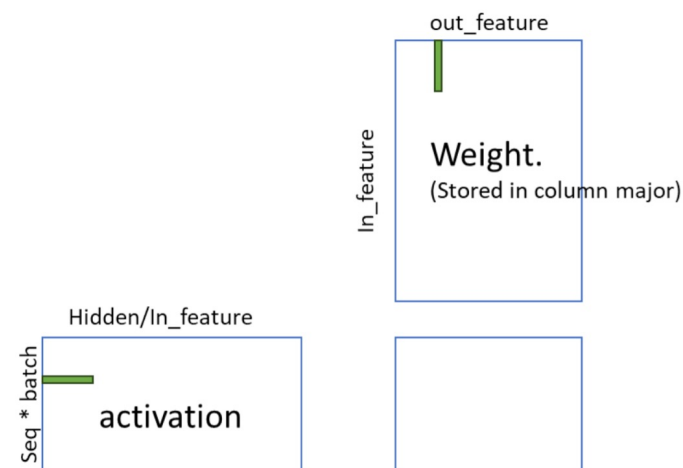
```
class RMSNorm(nn.Module):
    def __init__(self,
                 super().__init__()
                 self.weight = nn.Parameter(torch.ones(hidden_size, dtype=dtype))
                 self.variance_epsilon = eps

    def forward(
        self,
        hidden_states: torch.Tensor,
        residual: Optional[torch.Tensor] = None
    ) -> Union[torch.Tensor, Tuple[torch.Tensor, torch.Tensor]]:
        if IS_FLASHINFER_AVAILABLE:
            from ..custom_op import (flashinfer_fused_add_rmsnorm,
                                     flashinfer_rmsnorm)
            if residual is not None:
                flashinfer_fused_add_rmsnorm(hidden_states, residual,
                                             self.weight, self.variance_epsilon)
            return hidden_states, residual
        return flashinfer_rmsnorm(hidden_states, self.weight,
                                   self.variance_epsilon)
```

Extending Modules

Example: Support FP4 in Linear

- FP4: E2M1
 - 15 possible values: 0, ± 0.5 , ± 1.0 , ± 1.5 , ± 2.0 , ± 3.0 , ± 4.0 , ± 6.0
- Double quantization
 - 16 FP4 E2M1 values share a FP8 E4M3 per-block scaling factor
 - Per-block scaling factors share a FP32 per-tensor scaling factor
- Quantization is supported by ModelOpt
 - In FP4 ckpt, each linear module contains:
 - FP4 **weights**
 - **Per-block SFs** and a per-tensor SF
 - A **per-tensor SF** for the input activation
 - Activation quantization is dynamic
 - FP4 quantize: Given Fp16/BF16 input and the per-tensor SF, returns FP4 quantized output and per-block SFs
- Compared to FP8
 - ~2x memory saving (4.5 bit)
 - 2x TC performance



Extending Modules

Example: Support FP4 in Linear

- Declare Parameters of the module in `_create_weights` (called by `load_weights`)

```
class Linear(nn.Module):
    def _create_weights(self):

        # Quantized weights
        self.weight = Parameter(torch.empty(
            [self.out_features, self.in_features // 2],
            dtype=fp4_utils.float4_e2m1x2,
            device=device),
            requires_grad=False)

        # FP8 per-block scaling factors
        self.weight_scale = # ...

        # FP32 per-tensor global scaling factor = 448*6/amax_input
        self.input_scale = # ...
        self.inv_input_scale = #...

        # (amax_input*amax_weight) / (448*6*448*6)
        self.alpha = # ...

        self.profiler = torch.classes.trtllm.FP4GemmRunner.get_instance(
            self.dtype)
        self.needs_profiling = True
```

Extending Modules

Example: Support FP4 in Linear

- Load weight scales correctly
- E.g., for FP4 model.layers.0.self_attn.qkv_proj will also receive:
 - model.layers.0.self_attn.q_proj.weight_scale
 - model.layers.0.self_attn.q_proj.weight_scale_2
 - model.layers.0.self_attn.q_proj.input_scale
 - And the same for k_proj and v_proj...
- Block SFs are concatenated (and swizzled per the requirement of GEMM)
- Global SFs of QKV are ensured to be the same in ModelOpt

```
elif quant_mode.has_nvfp4():
    input_scale, weight_scale, alpha = load_weight_scales_nvfp4(
        weights,
        tp_size=self.tp_size,
        tp_rank=self.tp_rank,
        tp_mode=self.tp_mode)
    # Swizzle weight scales after concatenation
    weight_scale = torch.cat(weight_scale, 0)
    weight_scale = torch.ops.tensorrt_llm.nvfp4_block_scale_interleave(
        weight_scale)
    copy(self.input_scale, input_scale)
    copy(self.weight_scale, weight_scale)
    copy(self.alpha, alpha)
```

```
def load_weight_scales_nvfp4(weights: List[Dict],
                              tp_size: int = 1,
                              tp_rank: int = 0,
                              tp_mode: Optional[TensorParallelMode] = None)
    # For concatenated weights (qkv_proj / up_gate_proj), the global scale
    input_scale = None
    weight_scale_2 = None
    weight_scale = []

    for w in weights:
        if "input_scale" in w:
            if input_scale is None:
                input_scale = w["input_scale"][...]
            else:
                assert input_scale == w["input_scale"][...]
                ..., "The input_scale should be same for all the weights"
        if "weight_scale" in w:
            ws = load_weight_shard(w["weight_scale"], tp_size, tp_rank,
                                   tp_mode).contiguous()
            assert ws.dtype == torch.float8_e4m3fn # TODO: or e8m0 for
            weight_scale.append(ws.view(fp4_utils.float4_sf_dtype))
```

Extending Modules

Example: Support FP4 in Linear

- Implement `apply_linear`
- Dynamic activation quantization
 - Provide the static global SF and BF16 activations
 - Return FP4 quantized activations and FP8 block SFs
- Pick the best GEMM config according to profiling
- Launch GEMM with the following:
 - FP4 weights
 - FP8 weight block SFs
 - FP32 weight global SF
 - FP32 alpha (product of weight and activation global SF)

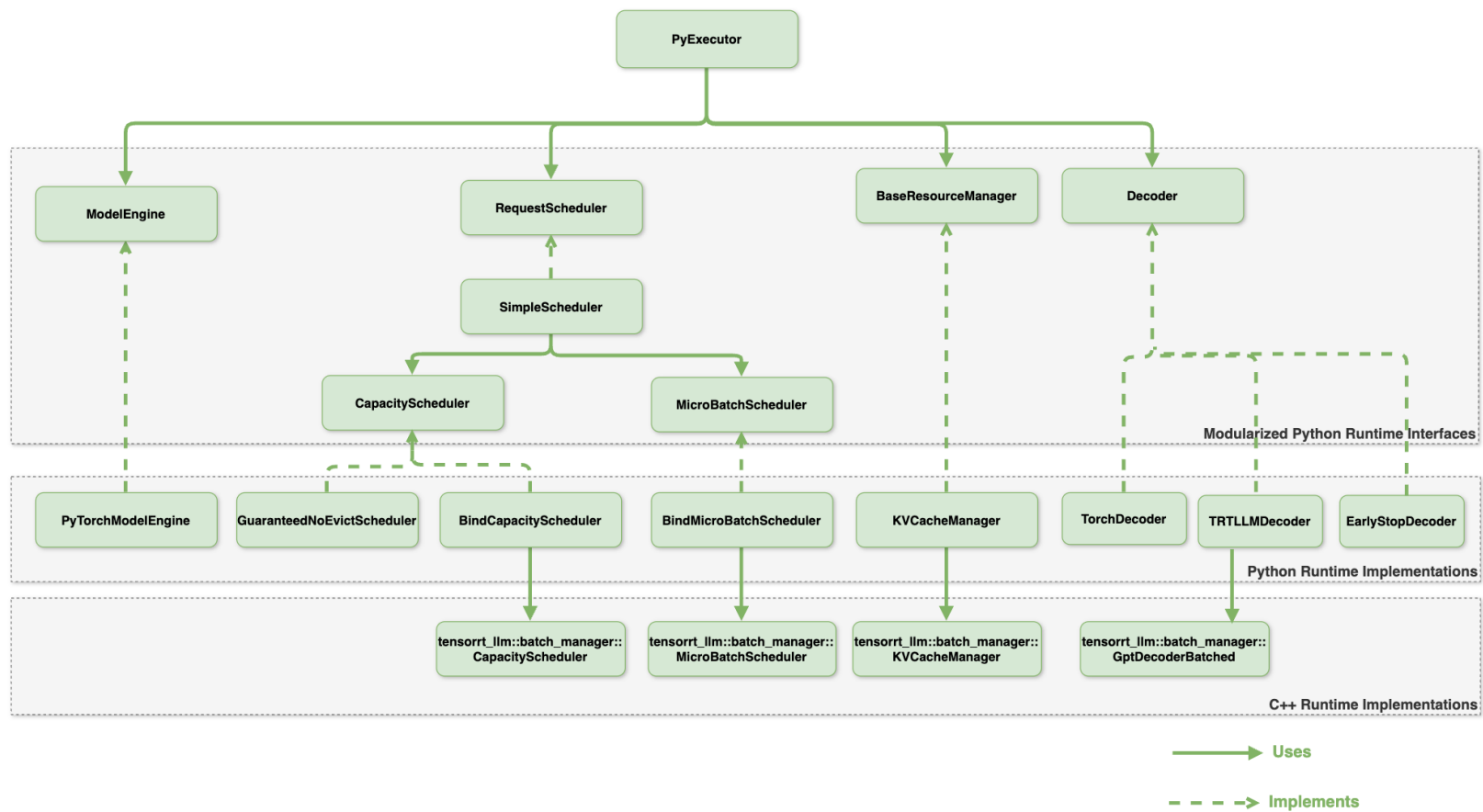
```
class Linear(nn.Module):
    def apply_linear(self, input, weight, bias):
        elif self.has_nv_fp4:
            m = math.prod(input.shape[:-1])
            n = self.weight.shape[0]
            k = self.weight.shape[1] * 2

            if self.needs_profiling:
                self.needs_profiling = False
                self.profiler.run_profile(n, k, fp4_utils.fp4_buckets)

            act_fp4, act_sf = torch.ops.trtllm.fp4_quantize(
                input, self.input_scale, self.scaling_vector_size, False)

            best_config_id = self.profiler.get_best_config_id(m, n, k)
            output = self.profiler.run_gemm(act_fp4, self.weight, act_sf,
                                           self.weight_scale, self.alpha,
                                           False, best_config_id)
```

Python Runtime Overview

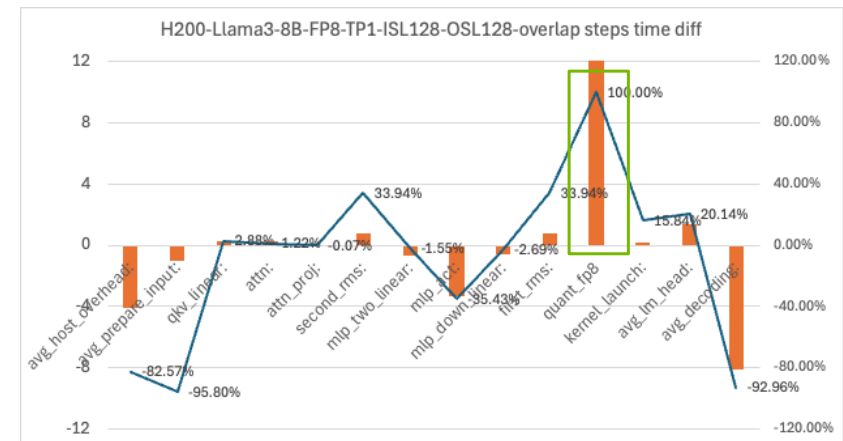
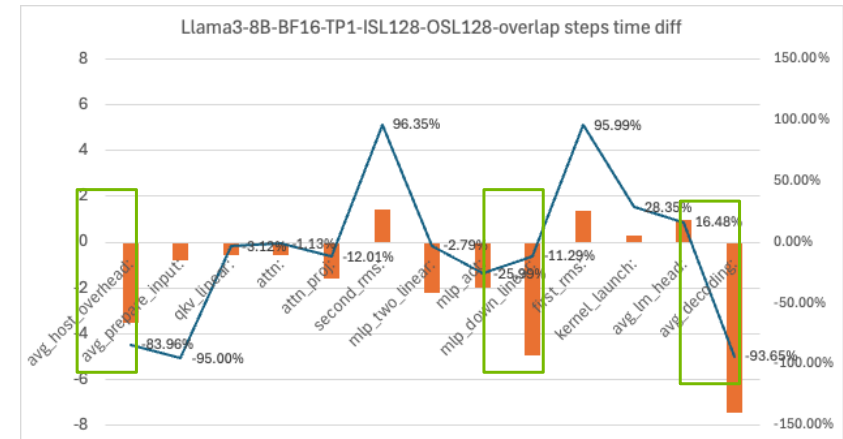
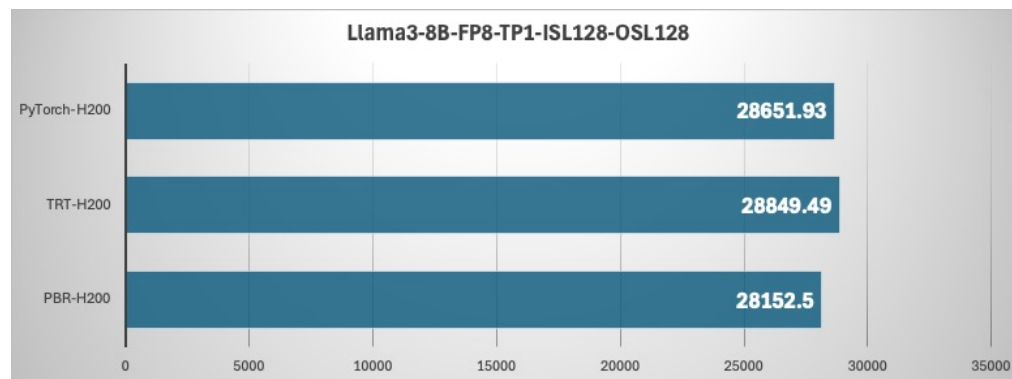
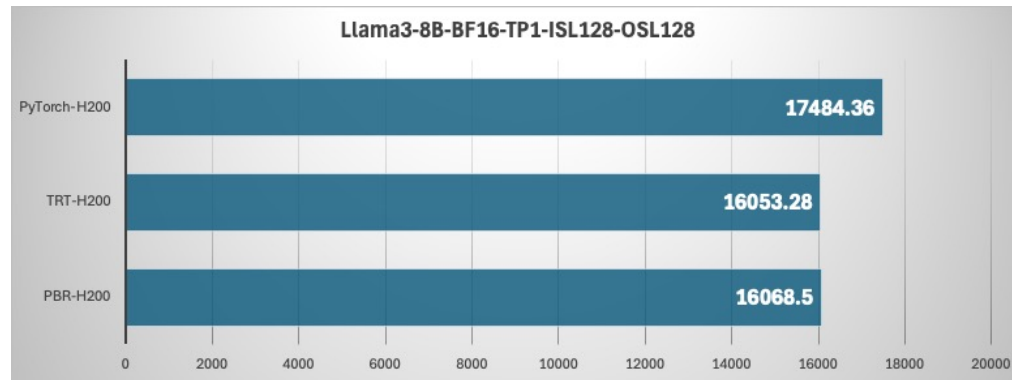


Modularized Runtime Modules

- Many runtime modules are customizable
- ResourceManager
 - KVCache Manager
- Scheduler
 - CapacityScheduler
 - requests -> fitting_requests, paused_requests
 - MicroBatchScheduler
 - fitting requests -> context_requests, generation_requests
- Currently many modules use C++ runtime components through Python bindings

Performance

vs TRT workflow





Agenda

- Introduction

- API & Ease-of-use

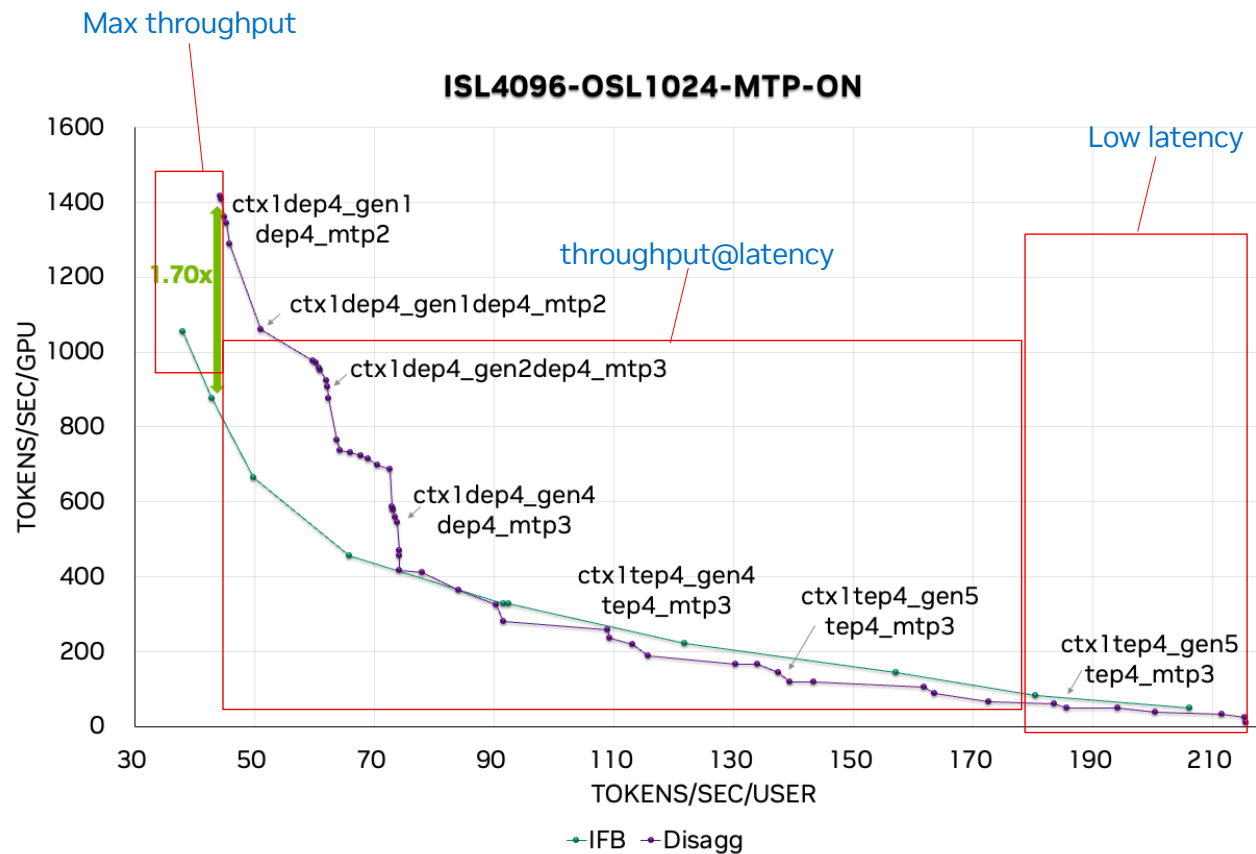
- PyTorch based Modeling&Python runtime

- **Performance Optimization**

- Future work

Performance Optimization

The landscape of inference performance



Performance Optimization

Methodology

- First step, understand the target workloads
 - Model structure, ISL/OSL, target concurrency, latency constraints, deployment environment
- Secondly, understand the target HWs
 - Computing power/memory bandwidth/Intra-node bandwidth/inter-node bandwidth/...
- Then,
 - Profile, locate the performance bottlenecks, evaluate based on the !/\$, and start the execution based on your judgement of priority
 - Pay attention to the complexity as to the SW architecture due to the optimizations, which can be accounted into the “\$” part
 - Repeat the cycle until you are satisfied

Performance Optimization

Max throughput

- Least constraints about latency, and the main goal is for throughput
- Fit for offline scenarios which only cares about TCO
- Saturate GPU as much as possible
 - Runtime optimization
 - Increase the effective Batch size
 - Distributed inference
 - Inflight batching/piggybacking
 - Chunked prefill
 - Attention DP(specifically for MQA and MLA)
 - Large-scale EP
 - Reduce non-computation overhead(Pipeline parallelism/low-precision communication)
 - KV Cache reuse
 - Kernel optimization(Attention/GEMM/Memory-intensive fusion)
 - Model optimization
 - Quantization
 - INT4 AWQ
 - Low-precision KV Cache cache(BF16 -> FP8)
 - Sliding Window Attention/StreamingLLM/Sparse Attention/...

Performance Optimization

Max throughput

- Using DS R1 as an example

Feature	TPS/GPU, ON	TPS/GPU, OFF	Impact
Attention DP	4122	1015	405%
Expert Parallel	4122	2884	142%
Overlap scheduler	4122	3945	4%
Cuda graph	4122	3357	22%
Cuda graph padding	4122	3660	12%
FP8 KV cache	4367	4122*	6%
TopK fusion	4122	3836	7.4%
Multi-stream optimizations	4122	3912	5.3%

FP8 KV cache. FP8 KV cache can also increase the concurrency a lot. 6% here is the same concurrency data

Refer [here](#) for more details.

Performance Optimization

Throughput@latency

- Balance between throughput and latency
- Usually the latency focus is on TPOT(or output token latency), still caring about TCO
- The more aggressive latency goal you want to hit, the more throughput you have to sacrifice
- Reduce latency interference
 - Dis-aggregated serving
- Reduce latency without sacrificing throughput too much
 - Large-scale EP
 - Speculative decoding
- Lots of techniques used for max throughput can also be applied here
 - Attention DP
 - Low-precision KV Cache
 - KV Cache reuse
 - ...
- What is not suitable for throughput@latency
 - IFB/Piggybacking
 - Chunked prefill

Performance Optimization

Throughput@latency
Dis-aggregated serving([tech blog link](#))

- Dis-aggregated serving
 - Motivation

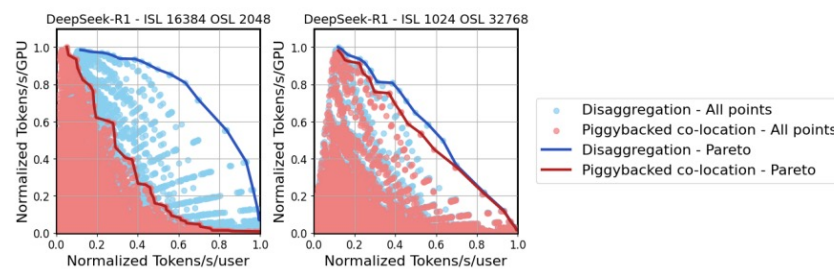
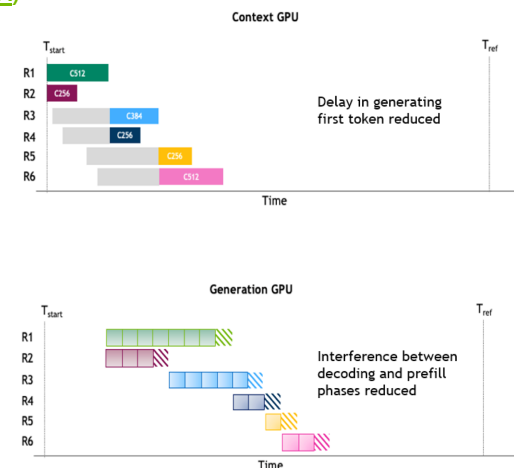
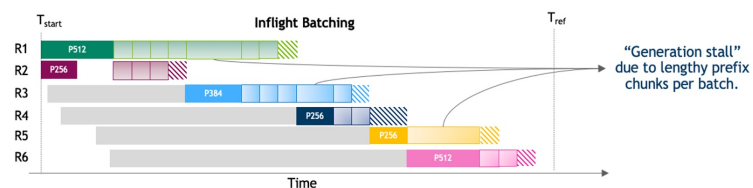


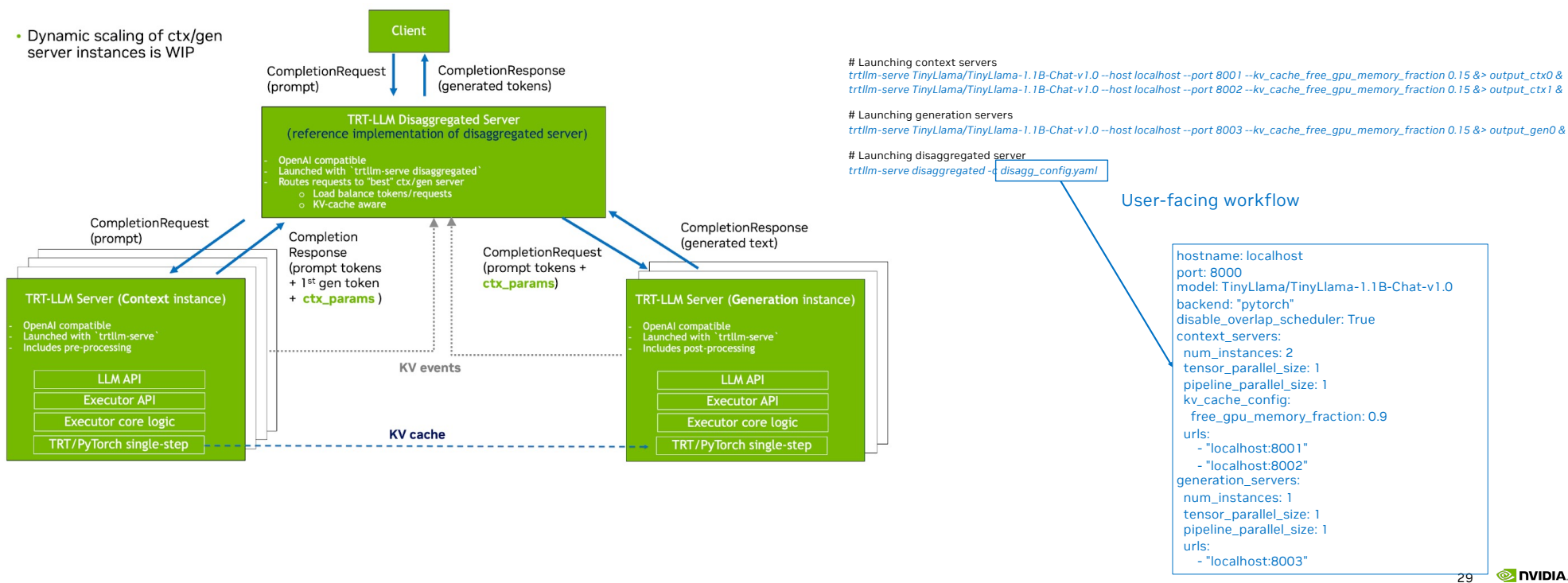
Figure 1: Throughput–interactivity Pareto frontier for DeepSeek-R1. The benefits of disaggregated serving vary with the target tokens/s/user (i.e., interactivity) and traffic patterns: (left) prefill-heavy vs. (right) generation-heavy traffic. Most results in this paper are presented in normalized form, as our primary objective is to convey trends rather than make specific performance claims.

Referred from "[Beyond the Buzz: A Pragmatic Take on Inference Disaggregation](#)"

Performance Optimization

Throughput@latency
Dis-aggregated serving

- trtllm-serve based workflow



Performance Optimization

Throughput@latency
Dis-aggregated serving

- KV Cache Exchange

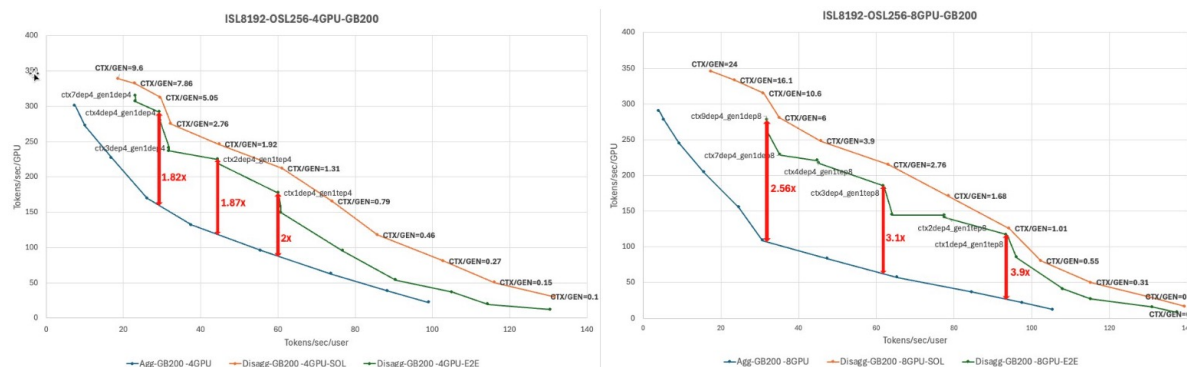


Performance Optimization

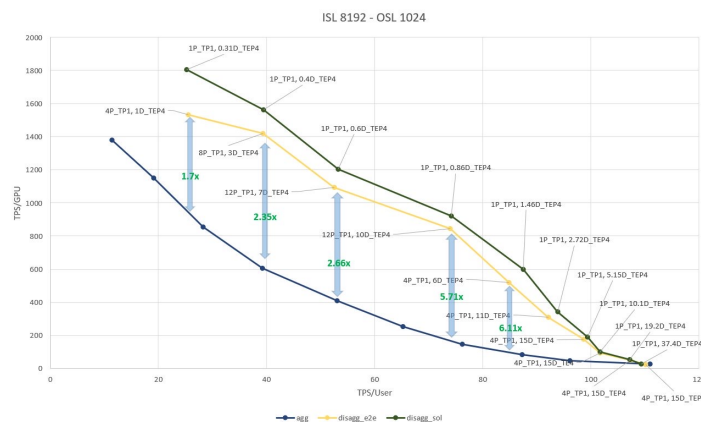
Throughput@latency
Dis-aggregated serving

- Performance result

DS R1



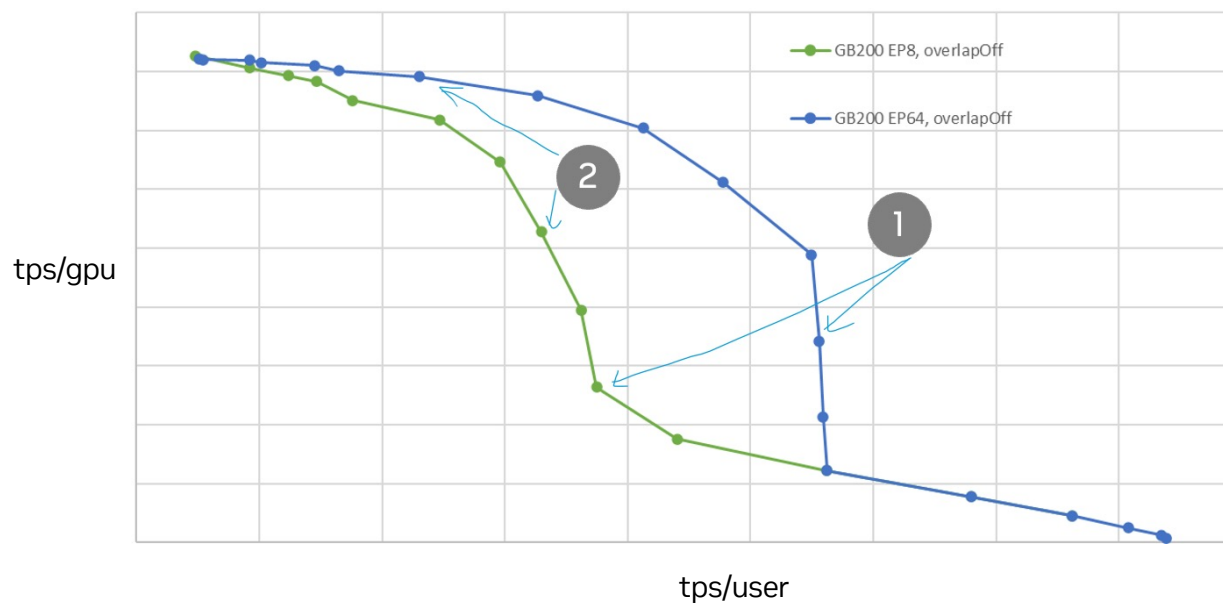
QWen3-235B



Performance Optimization

Throughput@latency
Large-scale EP(tech blog [link](#))

- Motivation(based on analytical model)



- 1 Reduces expert number per GPU, while keeping total compute ops of groupGEMM the same, thus proves the perf on BW bound range of groupGEMM(large tps/user range).
- 2 For memory capacity limited GPUs like H100, can achieve larger concurrency.

Performance Optimization

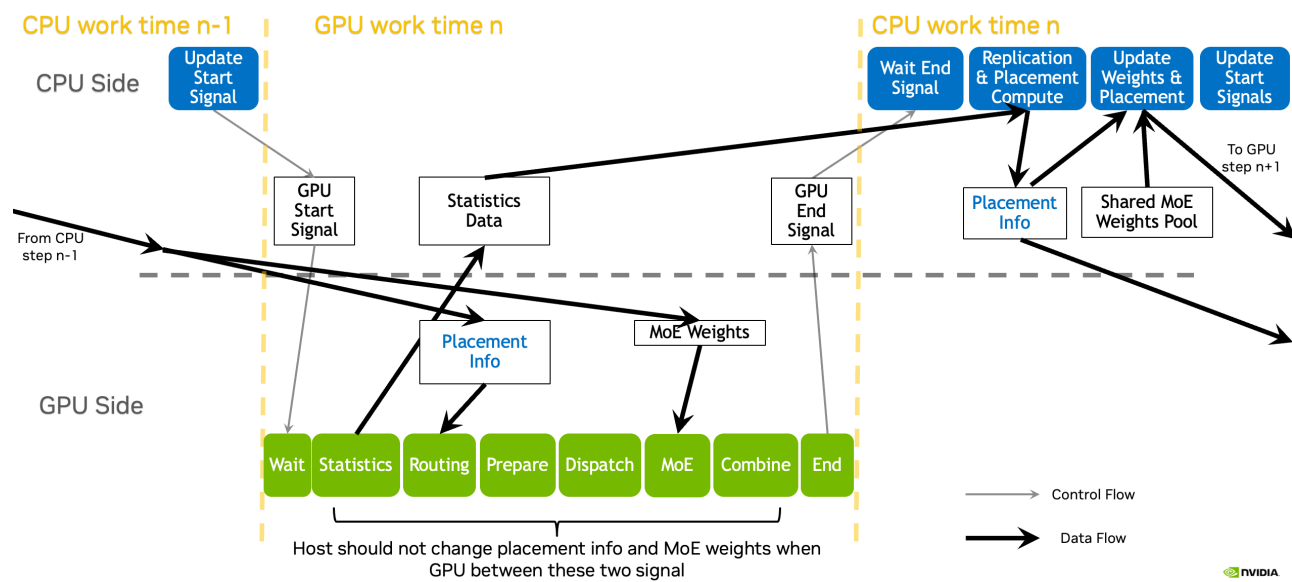
Throughput@latency
Large-scale EP

- Challenges
 - Performance implementation of expert communication kernels
 - NVL72's MNNVL helps
 - Solution: [Custom A2A leveraging MNNVL](#)
 - Proper handling of Load imbalance introduced due to larger EP size
 - Slowest rank may delay all other ranks([PR1](#), [PR2](#), [PR3](#), [PR4](#))
 - Solution: Online EPLB(Expert Parallelism Load Balancer)
 - Scale to more GPUs
 - All kernels matter to E2E performance should be scalable with EP size
 - Solution: Optimize the bottleneck to increase the EP scalability([PR1](#), [PR2](#), [PR3](#), [PR4](#), [PR5](#))
 - Latency is important
 - One main benefit region is throughput@latency, especially on B200/GB200
 - CUDA Graph should be supported

Performance Optimization

Throughput@latency
Large-scale EP

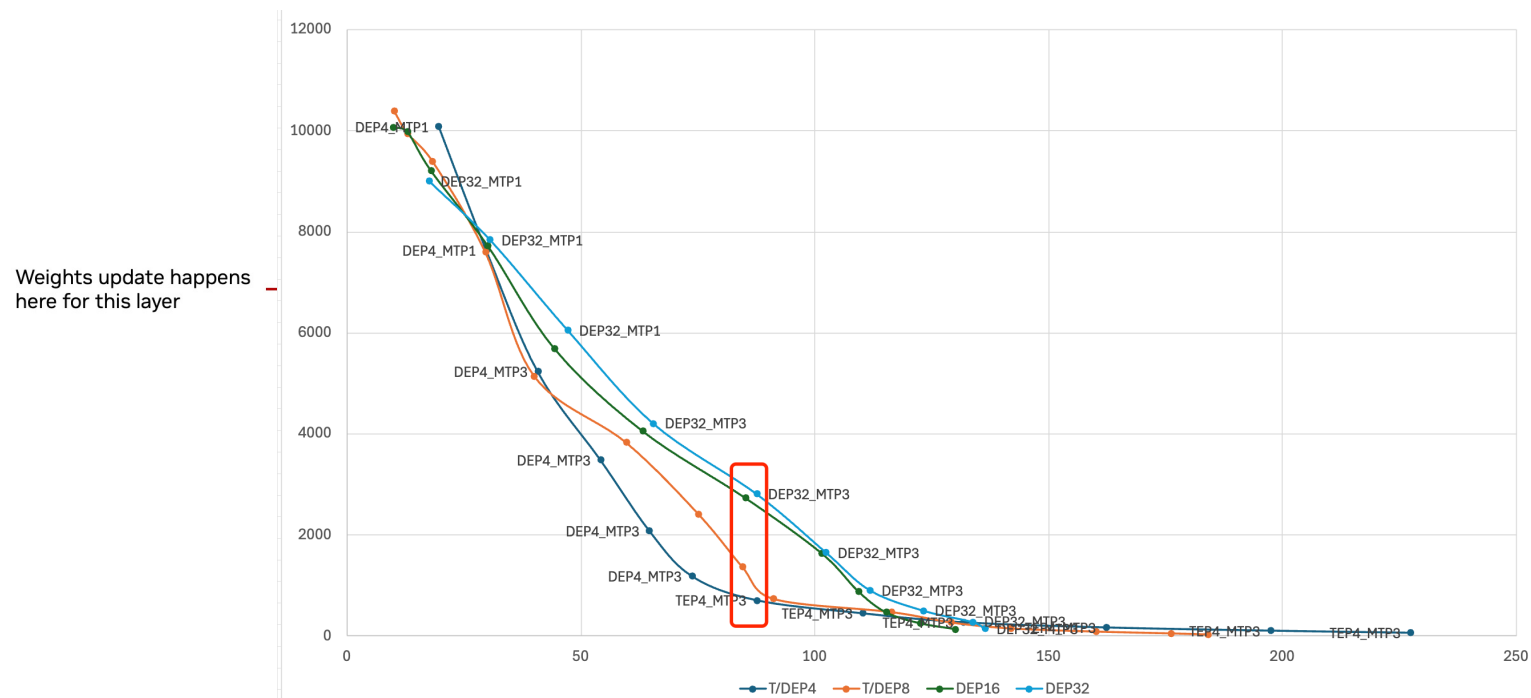
- Overall design



Performance Optimization

Throughput@latency
Large-scale EP

- Performance result



Performance Optimization

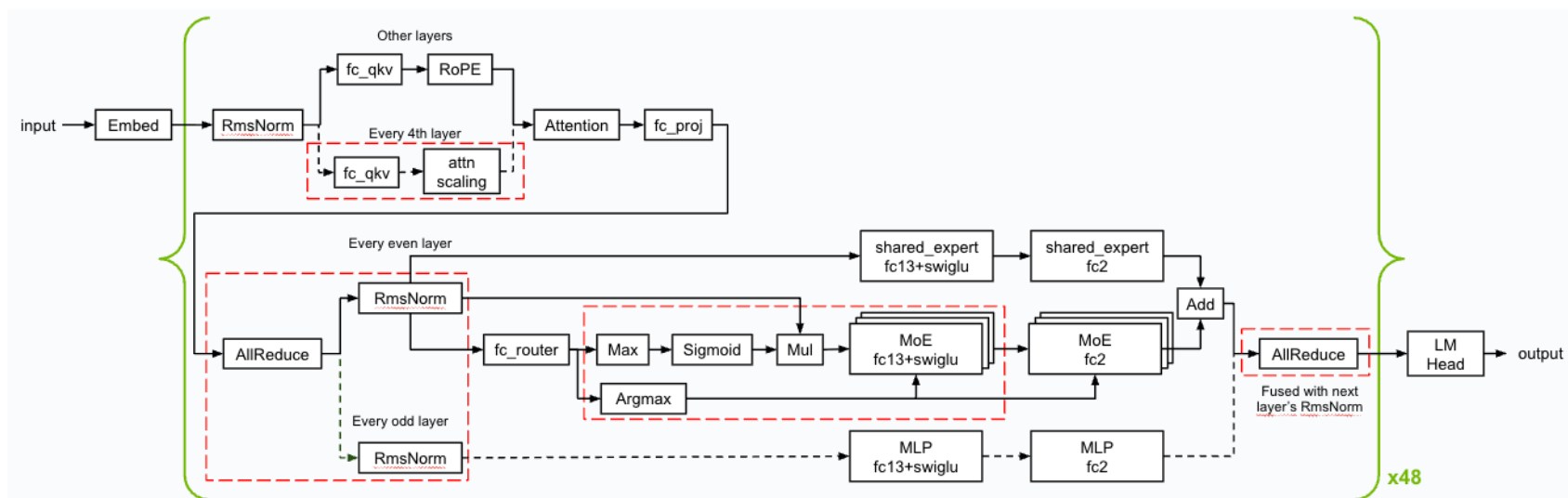
Low latency

- Focusing on the reduction of latency
- Not ideal for TCO, but can be essential for some critical scenarios
- Optimization focus
 - Aggressive kernel customization to map the workloads to more GPU computing units
 - More aggressive kernel fusion
 - Speculative decoding
 - Careful design of parallelism strategy

Performance Optimization

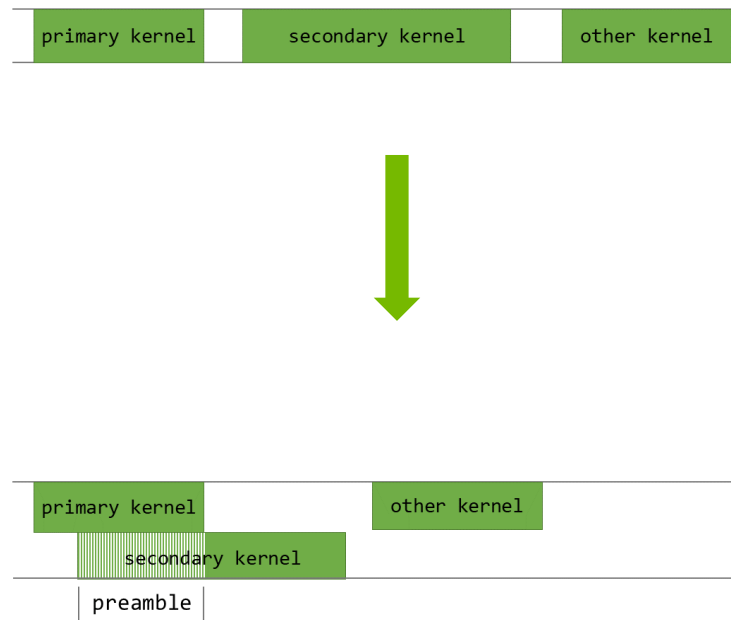
LLaMA4 as an example([blog link](#))

Kernel fusion for LLaMA4 Maverick



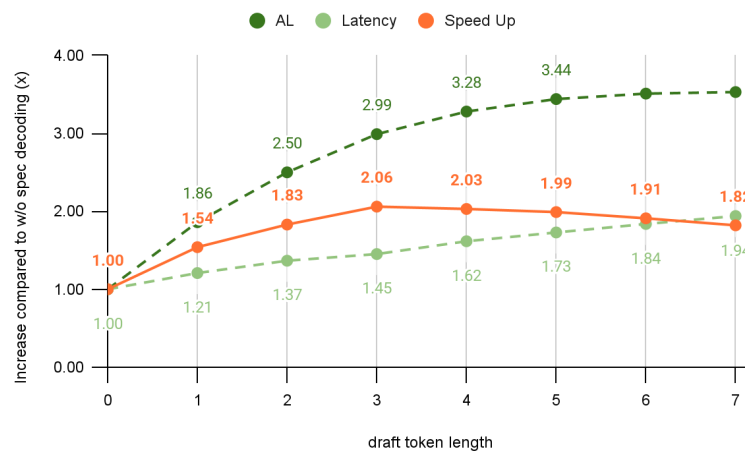
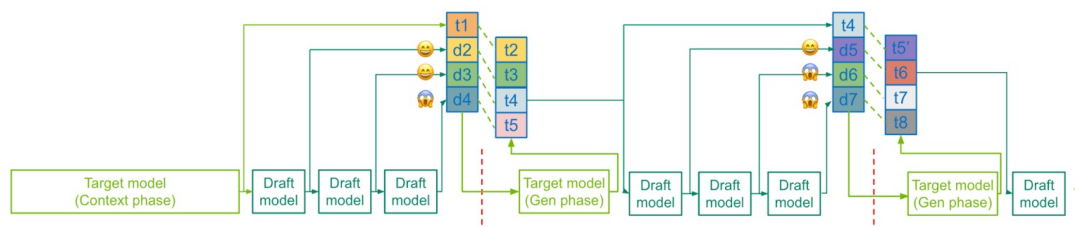
Performance Optimization

LLaMA4 as an example
Programmatic Dependent Launch(PDL)



Performance Optimization

LLaMA4 as an example
Speculative Decoding(Eagle-3 as an example)





Agenda

- Introduction

- API & Ease-of-use

- PyTorch based Modeling&Python runtime

- Performance Optimization

- **Future work**

Future work

- 1.0 release
 - PyTorch flow becomes the default one.
 - API backward compatibility
- Keeping pushing the performance boundary for key models, on key NVIDIA GPUs, for the entire performance landscape.
- Proper handling of various feature combinations to solidify the robustness and ease-of-use.
- Post-training support.
- Constantly sharing more inference optimization experience to the community to lift the performance boat on NVIDIA GPUs.

Resources

- Tech blog
 - [Pushing Latency Boundaries: Optimizing DeepSeek-R1 Performance on NVIDIA B200 GPUs](#)
 - [DeepSeek R1 MTP Implementation and Optimization](#)
 - [Optimizing DeepSeek R1 Throughput on NVIDIA Blackwell GPUs: A Deep Dive for Developers](#)
 - [Scaling Expert Parallelism in TensorRT-LLM \(Part 1: Design and Implementation of Large-scale EP\)](#)
 - [Disaggregated Serving in TensorRT-LLM](#)
 - More to come
- Live stream
 - [Beyond the Algorithm with NVIDIA: The New PyTorch Architecture for TensorRT-LLM](#)
 - [Beyond the Algorithm with NVIDIA: TensorRT-LLM Goes GitHub First](#)
 - [DeepSeek R1 performance optimization to push the latency performance boundary](#)
 - [Implementation and optimization of MTP for DeepSeek R1 in TensorRT-LLM](#)
 - More to come
- [Roadmap](#)



<https://github.com/NVIDIA/TensorRT-LLM>

Join the Community on TensorRT-LLM Wechat group



请添加小助手微信并在申请中
备注“TRT-LLM”入群